Unreal VFX

A high-level view guide of how parts connect together.

Welcome!

This is an introductory guide to connecting various parts of unreal engine together for VFX. This is not an in-depth tutorial on any one of the covered components but rather how they could work together in logical ways.

We will discuss Blueprints, Dynamic Materials, and Niagara Particles. With an emphasis on how they work together and when to use one or the other.

Blueprints - Overview

In unreal, blueprints are the glue that holds things together. Blueprints are groupings of components (smaller pieces) and allow code to be written to determine behavior.

I am mostly going to be using examples of blueprint actors that you can place in the world. These mechanics also translate to pawns and other blueprint types.

Blueprints allow **event-driven programming**. This means that when something happens, we can react. Examples:

> *When the W key is pressed, the Player moves forward.*
>
> *When two objects collide, we spawn an explosion*
>
> *On tick, increment a counter. When counter equals 100, spawn an enemy*

The event driven nature allows you to define how logic in your scene works. All of the examples above involved multiple different components and types. Blueprints are a way to store connections between parts and define the logic for how they should interact.

If you want multiple systems to interact, you want to do it in Blueprints.

Here are some examples of things I have made. There full write ups are listed elsewhere but just as an example.

- A Blueprint forcefield hit by a Blueprint bullet spawns a Niagara impact effect.
- A Blueprint magic circle that causes the underlying Niagara system to turn on a snow emitter when the player enters. Entering the circle also causes the player to change its overlay material to frost and turn on a Niagara system to spawn snow above the player. (Here I am interacting with both the collided object and the other object that did the colliding).

Below I'll take you through Materials and Niagara and how they could interact with blueprints. Once they have interacted with blueprints, you can use that information to do anything you want including influence other components.

Materials

You probably already have some experience with Unreal Materials. These are how we put colors and textures onto static mesh. Materials can be a Textures mapped perfectly to the static meshes UV coordinates (coordinates in 2d space) or generated algorithmically using techniques used in shader code (distance to points, noise maps, ectara). Regardless, Materials take in the UV coordinates of a mesh and tell the engine what color should be at every pixel and how it should react to light.

Dynamic Materials

What's exciting about Materials is that they can be parametrized. If you start adding parameters to your materials, then instances of them can share commonalities but allow customization. That works great in the editor. Making master materials, streamlines setting up materials that are similar to each other.

But dynamic material instances **can work at runtime.** So you can create materials that change while the game is running. Say for a force field to go down and have a fizzle out effect or an overlay glow effect to get progressively brighter.

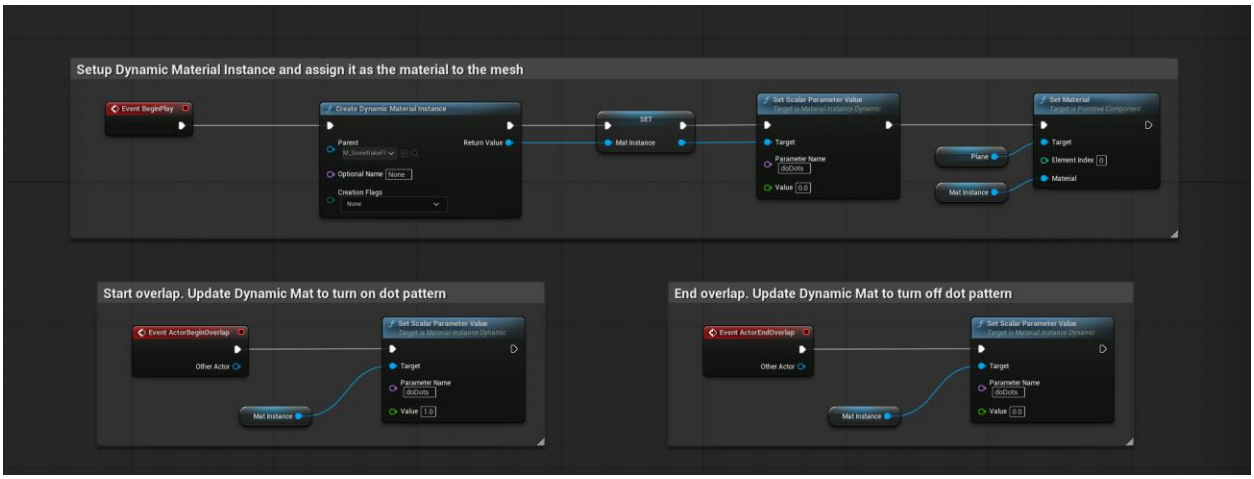
Without dynamic materials, we could achieve these effects by switching out the material, but we can do it directly using parameters. This cleans up our code and allows us to create more complex materials.

Dynamic Material Instances are controlled by Blueprints. So, you would make your material, give it parameters, with clear names, and then make a blueprint that controls this objects material. The Blueprint would create a Dynamic Material Instance and can then be modified.

The workflow here is Blueprints control Dynamic Material Instances.

Instructions

1. Create a Material
    a. Right Click any data type node and click "Convert to Parameter".
    b. Name that Parameter something specific and relevant. Good practice is to not have any spaces
    c. Apply and save
2. In your Blueprint, create a Dynamic Material Instance. save as local variable
3. Assign parameters on your instance. Anything you don't explicitly define will default to its base value (set in the material itself)
4. Assign Dynamic Material to a mesh in Blueprint
5. Elsewhere on an event, update the parameter in the local variable
6. The static mesh will show the changes you made



Example Update material variable doDots when you stand in the overlap area of the blueprint. This code simply turns on the glowing lights on the snowflake texture.

Side note: Material Complexity

This is going to be a very high overview of complexity, but it's worth touching on. Materials tell the engine what to render every frame. The more complicated a material is the longer it takes the GPU to compute. This is something you're mostly going to pay attention to if you're optimizing your game for lower end graphics cards.

In general, more nodes in a material and particularly more texture lookups are more expensive.

In general noise nodes are more expensive than texture lookups.

In general, translucent materials are more expensive than solid ones. This is because the engine must then compute more than one shader per pixel. The transparent material pixels and then whatever is behind it. This keeps going if what's behind it is another transparent material. The more transparent objects are on the screen the more complex your scene is going to be.

Masked materials, on the other hand are less expensive than translucent materials. This makes sense, masked materials allow opacity to be either zero or one. You either compute the current pixel or discard it and compute the below. Masked materials are great for small transparent objects where detail doesn’t matter so much. This is great for many Niagara particles, especially those far away.

You can see the material complexity of your scene by hitting Alt 8 (MAC) or by navigating to views -> shader complexity.

Shader complexity is mostly important when something is going wrong or you are developing mobile devices or similar low graphical systems.

Using Niagara for projectiles; thoughts...

Niagara is meant to be used as a visual display. If I was making an explosion. I might do it all in Niagara. But if I was making a spray bullet effect, I wouldn't have the effect logic be in Niagara, just the bullet. This means I would probably make a different Niagara instance per bullet or simply use a mesh. The actual logic I would do in Blueprints using a line trace. There isn't one way only to do this but there are simpler ways and more complicated. If I don't care that the bullet spray hits a specific point, I might do it all in Niagara. This would be great for background effects; it wouldn't be so great for targeting an enemy with a scope rifle.

Remember visual effects is all smoke and mirrors. It doesn't matter if the Niagara bullet is not actually causing the collision as long as you don't break the illusion that it is. You pick what the most important elements of your illusion are and work from that.

Now Niagara does have collision events. These are mostly internal events and can be used to emit new particles at a location. Say a small explosion or dent decal. So Niagara can be great for simulating the effects at scale.

I could use the same materials and objects as my background smoke and mirrors Niagara system on my main characters weapon. That way it would look the same but would give me logic to work with. (I hit the target, it takes damage).

You *can* use the collision events to deal with damage, as you can pass limited collision data back up to the blueprint. Just know that in a multiplayer game the Niagara randomness is client side and thus won't be consistent across clients.

So why wouldn't I do everything in Blueprints? Well, Niagara is good at what it does do: efficiently spread lots of visual particles. If I want lots of laser fire in the background, working with Niagara is the easiest way to do that. I don't care what they hit, I just want there to be lots of them. Also, if I want a complicated fireball effect, I would make that in Niagara. Its one hit check, which I'd do in blueprints but I if I want that fireball to look complicated I'd probably do it in Niagara.

Always remember, the effect you are making impacts what tool you want to use.

Blueprints interacting with Niagara

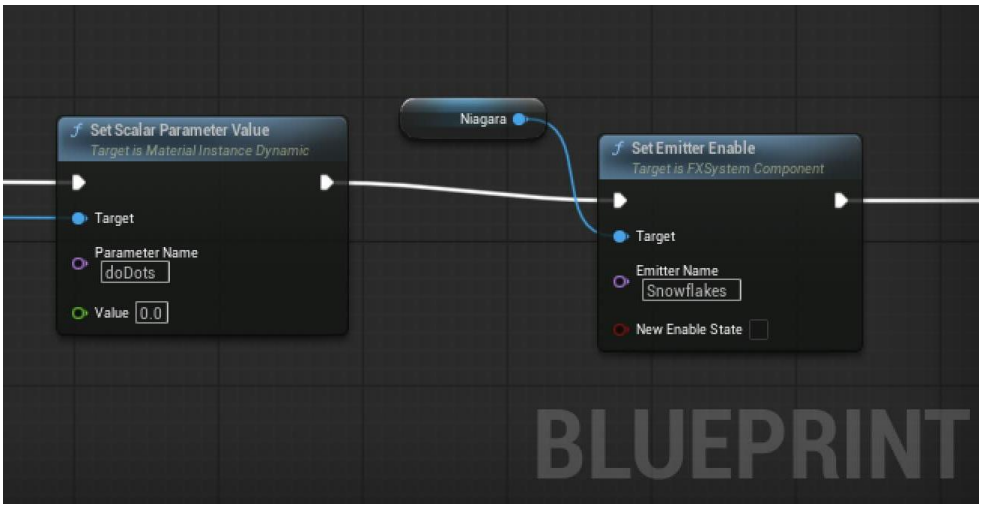
Niagara Systems can both receive information from Blueprints and send it. Most of the time receiving information will be the most useful.

Sending Data from Blueprints to Niagara

Built in is the ability to turn on and off any individual emitter.

Niagara also allows custom parameters, you can edit these at runtime using Blueprints

This could change a material, the number of particles, the speed, really any value of the particle.



Blueprint nodes to change a Niagara parameter and toggle an emitter.

On Sending Data from Niagara to Blueprints

You can send limited collision data upwards.

Specifically three vectors. Those usually are:

- Location
- Velocity
- Particle Color or another particle attribute

But you don’t have access to the colliding actor or collision normal vector.

You could use this for particle collisions. You would need to do a sphere trace after collision on the blueprint side. This requires you to search for collisions twice, which is a bit redundant, but that's what you’d need to do to use Niagara particles to generate hit events. The normal way would be to have a blueprint generating hit events with a Niagara particle following it for visuals.

Resources

Dynamic Materials Resources

[UE DOCS: Material Instances](https://dev.epicgames.com/documentation/unreal-engine/instanced-materials-in-unreal-engine?lang=en-US)

[UE DOCS: Creating and Using Material Instances]((https://dev.epicgames.com/documentation/unreal-engine/creating-and-using-material-instances-in-unreal-engine?application_version=5.7)

Niagara Resources

Connection Unreal to Blueprints

[UNREAL DEC COMMUNITY SITE: Blueprint Module: Controlling Niagara Particles (Part 1)](https://dev.epicgames.com/community/learning/tutorials/vaEw/unreal-engine-blueprint-module-controlling-niagara-particles-part-1)

Shows how to activate Niagara systems programmatically.

[YOUTUBE: UE4 - Niagara Export Particle Data to Blueprint – Collision, gameDev Outpost](https://www.youtube.com/watch?v=aA_8NLzbUTA)

[UNREAL FORUMS: Blueprint Collision events from Niagara](https://forums.unrealengine.com/t/blueprint-collision-events-from-niagara/456264)

Creating Niagara particles

[YOUTUBE: UE5 l Poison Bubble Effect l Niagara VFX Tutorial l UE5, Coreb Games](https://www.youtube.com/watch?v=FjAEYMIjMNM)

[YOUTUBE: Unreal5 Niagara VFX: Crash-Course (INTRODUCTION!), Royal Skies](https://www.youtube.com/watch?v=8k0vuDD_cE0) Spark effect and how to read the editor.

[YOUTUBE: UE5 l How To Create Magic Circle Niagara Effect l VFX Tutorial l Unreal Engine 5](https://www.youtube.com/watch?v=KEPaj9k9n7Q)

[YOUTUBE: Unreal Engine 5 - Stylized Smoke VFX - Niagara Tutorial, Gabriel Aguiar Pd.](https://www.youtube.com/watch?v=HRagD5L-WF8)

[YOUTUBE: Unreal Engine 5 Snow Particle Tutorial, CodeLikeMe](https://www.youtube.com/watch?v=sUW3or-YCrk)

[YOUTUBE: Advanced Reactive Force Fields - with Niagara in UE5, UnrealCG](https://www.youtube.com/watch?v=8QI6WLbrFEY)

[YOUTUBE: Create a Healing Spell in UE5.5: Ultimate Niagara VFX Tutorial, CGHOW](https://www.youtube.com/watch?v=dhgRINBVOZw)

Optimization

[Complete Guide to Niagara VFX Optimization in Unreal Engine, MORE VFX Academy](https://morevfxacademy.com/complete-guide-to-niagara-vfx-optimization-in-unreal-engine/)

[Niagara VFX Optimization: Part 2 — Profiling, Scalability, and Performance Tips, MORE VFX Academy](https://morevfxacademy.com/niagara-vfx-optimization-part-2-profiling-scalability-and-performance-tips/)

How to read the profilers. You might not need these for a while, but if you do here is a guide.

Breakdowns of different effects

[YOUTUBE: Breakdown - Spell Breaker FX in UE5.3 Niagara](https://www.youtube.com/watch?v=6AJkngIqLA4)

[YOUTUBE: [Noara] Waterfall, Cool Cat VFX](https://www.youtube.com/watch?v=XIrebDJUzb4&)